Escales — mini-rapport du projet NoSQL

Objet de l'application

Escales est une application de démonstration de location de logements. Un utilisateur peut consulter les annonces, effectuer une recherche par ville, capacité, prix ou mots-clés, créer et modifier une annonce, puis enregistrer une demande de réservation depuis l'interface ou l'API. Le projet illustre la séparation de responsabilités entre deux services ainsi que la propagation asynchrone d'événements.

Services et agrégats

Service Locations

Le Service Locations est responsable des données métier source. MongoDB stocke les agrégats Listing et Reservation dans des collections distinctes.

• Listing est racine par listingId. Il contient le titre, la description, l'hôte, la ville, le pays, la capacité, le prix, le statut, une version et les dates de création/modification. Les changements de cet agrégat sont enregistrés dans MongoDB.
• Reservation est racine par reservationId. Il référence l'annonce par listingId et mémorise le voyageur, les dates, le montant calculé et son statut. Elle a son propre cycle de vie et sa propre version.

L'outbox d'événements en attente est conservée dans le document de l'agrégat. Cela rend atomiques l'écriture d'un changement et l'ajout de son événement, sans exiger un replica set MongoDB dans cet environnement de démonstration.

Service Recherche

Le Service Recherche consomme Redis Streams et maintient une projection Redis des annonces, optimisée pour la lecture. Il possède aussi son agrégat SearchQueryStats, stocké sous forme de compteurs Redis par ville. Il ne modifie jamais les agrégats source dans MongoDB.

Bases NoSQL et justification

MongoDB

MongoDB est orientée documents. Les annonces et réservations sont des documents JSON/BSON et leurs champs peuvent évoluer avec les besoins de l'application.
La lecture et la mise à jour d'une annonce sont centrées sur son identifiant.
Des index uniques protègent listingId et reservationId, tandis qu'un index ville/statut facilite les lectures de démonstration.

Redis

Redis complète MongoDB avec plusieurs structures adaptées aux usages de lecture : chaînes JSON pour les annonces projetées, ensembles pour indexer les annonces par ville, hash pour les versions et statistiques, et Redis Streams pour transporter les événements. Cette séparation évite de solliciter MongoDB pour chaque recherche et permet de présenter une cohérence à terme.

Échange d'événements et réplication

Après création ou modification d'une annonce, le Service Locations enregistre un événement ListingPublished, ListingUpdated ou ListingDeleted dans l'outbox MongoDB. Un worker publie ces faits dans le stream Redis domain-events. Chaque événement comporte un eventId, un type, une date, l'identifiant et la version de l'agrégat ainsi qu'un instantané de l'annonce.

Le Service Recherche consomme ce stream dans le groupe search-projection. Il met à jour la projection Redis avec un script atomique qui refuse les versions anciennes et les doublons. Il acquitte ensuite l'événement. Les messages invalides sont consignés dans un flux de lettres mortes et les erreurs sont visibles dans les journaux.

L'identifiant d'événement est également enregistré dans Redis lors de la publication afin qu'une reprise du worker après un XADD ne crée pas une seconde entrée. La livraison est donc au moins une fois, avec publication et projection idempotentes. La projection est éventuellement cohérente : un changement peut ne pas apparaître immédiatement dans les résultats de recherche.

Démonstration

Depuis la racine du dépôt, lancer docker compose up --build, puis ouvrir http://localhost:8001/. Le site web montre les annonces de la projection, permet d'en publier une dans MongoDB et d'envoyer une demande de réservation. Après quelques instants, la recherche Redis doit la retourner. Les endpoints /events, /projection et /stats
permettent d'observer le transport, le réplicat et l'agrégat de recherche.
Les documentations OpenAPI sont exposées sur /docs par chacun des services.

Limites et suites possibles

Ce projet est un MVP pédagogique. Il ne fournit pas d'authentification, d'autorisation, de paiement, de réservation atomique de nuits ni de configuration de production. Les identifiants hôte/voyageur servent à la démonstration. La projection est stockée dans Redis et peut être reconstruite en rejouant le stream ; en exploitation, il faudrait définir sa politique de rétention, la surveillance et la sauvegarde des événements, la taille maximale de l'outbox embarquée et une stratégie de gestion des conflits plus élaborée.

Conclusion

Le projet satisfait le scénario NoSQL demandé avec deux services, deux bases NoSQL, des agrégats indépendants et un réplicat de lecture actualisé par des événements inter-services. MongoDB reste propriétaire des annonces et réservations ; Redis fournit le transport événementiel, le réplicat de recherche et ses statistiques.